

Complete Step-by-Step Guide to Using GitHub Classroom

This guide provides detailed instructions for setting up and managing 10 homework assignments using GitHub Classroom for a statistical computing course.

Initial Setup Phase

Step 1: Create a GitHub Organization

1. Log in to your GitHub account
2. Click on your profile picture in the top-right corner
3. Select “Your organizations” from the dropdown menu
4. Click the green “New organization” button
5. Select the “Create a free organization” option
6. Enter an organization name (e.g., “stat-computing-fall2025”)
7. Enter your contact email
8. Select “My personal account” for the organization owner
9. Verify that you’re not a robot if prompted
10. Click “Next”
11. On the “Welcome to GitHub” screen, select “Skip this step” (you can add members later)
12. You now have a GitHub organization for your course

Step 2: Connect to GitHub Classroom

1. Go to classroom.github.com
2. Click “Sign in” and authenticate with your GitHub account
3. If this is your first time, click “Authorize GitHub Classroom”
4. Click “Create your first classroom” or “New classroom”
5. Select the organization you just created
6. Confirm by clicking “Grant” to give GitHub Classroom access to your organization
7. Enter a name for your classroom (e.g., “Statistical Computing Fall 2025”)
8. Click “Continue”
9. Decide whether to require students to accept invitations before they can access assignments
 - For most courses, select “Yes” so you know who has joined
10. Click “Continue”
11. On the “Integrate your learning management system” page, you can either:
 - Set up LMS integration (Canvas, Moodle, etc.)
 - Click “Skip” to set it up later
12. Your classroom is now set up

Step 3: Add Teaching Staff to the Organization

1. Go to your organization page (github.com/your-org-name)

2. Click on “People” in the top navigation
3. Click “Invite member”
4. Enter your TA’s GitHub username or email
5. Select the role “Owner” (for full administrative access)
6. Add a message if desired
7. Click “Send invitation”
8. Repeat for any additional TAs or instructors
9. The invited TA will receive an email and must accept the invitation

Step 4: Create a Template Repository for Assignments

1. Go to your GitHub profile (not the organization)
2. Click the “+” icon in the top-right corner and select “New repository”
3. Name it “assignment-template”
4. Add a description: “Template repository for statistical computing assignments”
5. Select “Public” visibility
6. Check “Add a README file”
7. Add a .gitignore file for R
8. Select a license (e.g., MIT License)
9. Click “Create repository”
10. Once created, click “Settings” in the repository
11. Scroll down to the “Template repository” section
12. Check the box to make this a template repository
13. Create a basic structure in this repository:
 - Create a folder called “tests” for test files
 - Add a sample assignment file
 - Add a sample test file
 - Add any other common files needed for all assignments

Creating and Setting Up Assignments

Step 5: Create Assignment #1: Hello World in R

1. Go to your GitHub Classroom
2. Click on the classroom you created
3. Click “Create assignment”
4. Select “Individual assignment”
5. Enter “Assignment 1: Hello World in R” as the assignment title
6. Enter a deadline if desired
7. Choose a repository visibility:
 - “Private” is recommended so students can’t see each other’s work
8. Enable the option “Add a template repository to give students starter code”
9. Select your template repository from the dropdown
10. Decide whether students can use an online IDE
11. Click “Continue”

12. On the next screen, you can set up autograding:
 - Toggle “Enable autograding” to ON
 - Click “Add test”
 - Select “Run command” from the dropdown
 - Enter the command to run your tests (e.g., `Rscript tests/test_hello_world.R`)
 - Set the timeout (e.g., 5 minutes)
 - Set the number of points (e.g., 40)
 - Click “Save test case”
13. Click “Create assignment”
14. You’ll now see an invitation link - copy this to share with your students

Step 6: Customize the Repository for Assignment #1

1. After creating the assignment, click “Go to starter code repository”
2. This opens the template repository you created
3. Use the GitHub web interface or clone locally to:
 - Update the README.md with specific instructions for Assignment 1
 - Create or modify `hello_world_template.R`
 - Create `tests/test_hello_world.R` with test cases
4. For the test file, create R code that:
 - Sources the student’s `hello_world.R` file
 - Checks if it prints “Hello, World!”
 - Checks if it creates a variable called “name”
 - Checks if it prints a personalized greeting
 - Outputs a score based on these checks

Step 7: Repeat for Remaining Assignments

For each of the remaining 9 assignments:

1. Follow Step 5 to create a new assignment in GitHub Classroom
2. Name it appropriately (e.g., “Assignment 2: Data Visualization in R”)
3. Use the same template repository
4. Set up autograding specific to the assignment
5. Follow Step 6 to customize the repository for that specific assignment
6. Create appropriate test files for each assignment
7. Ensure each assignment’s README clearly explains the requirements

Student Enrollment and Assignment Distribution

Step 8: Enroll Students in the Classroom

1. Go to your GitHub Classroom
2. Click on the classroom you created
3. Click on “Students” in the top navigation
4. You have two options:
 - Manually add student names/identifiers

- Use the “Import from CSV” option if you have a class roster
5. Click “Create roster”
 6. You’ll now see your class roster

Step 9: Distribute Assignment #1

1. Copy the invitation link for Assignment 1
2. Share it with students through your learning management system or email
3. When students click the link, they will:
 - Log in to GitHub (or create an account)
 - Link their GitHub account to their roster identifier
 - Accept the assignment
 - Get a personal copy of the assignment repository
4. Students will work in their personal repositories and push changes
5. Track student progress on the assignment page in GitHub Classroom

Grading and Feedback

Step 10: Access and Grade Assignment Submissions

1. Go to your GitHub Classroom
2. Click on the classroom
3. Click on the assignment you want to grade
4. You’ll see all student submissions with:
 - Submission status
 - Autograding results
 - Links to their repositories
5. Click on a student’s repository to review their work
6. Review the code in detail and leave comments:
 - Click on specific files to review
 - Click on specific lines and use the “+” icon to add line-specific comments
 - Consider code style, documentation, and solution approach
7. Calculate the final grade:
 - Start with the autograding score
 - Add your manual assessment score
 - Record the final grade

Step 11: Provide Feedback and Grades

1. For each student submission:
 - Add a summary comment on the latest commit or create an issue
 - Include strengths, areas for improvement, and the grade breakdown
 - Use GitHub’s review feature for more formal feedback:
 - Go to the “Pull requests” tab
 - The student may have created a PR, or you can create one
 - Click “Review changes”

- Enter your feedback
 - Select “Comment”, “Approve”, or “Request changes”
 - Submit review
- 2. Update the grade in GitHub Classroom:
 - Go back to the assignment in GitHub Classroom
 - Find the student’s submission
 - Click on the grade
 - Enter the final grade
 - Click “Save”
- 3. Export grades if needed:
 - On the assignment page, click “Download grades”
 - This creates a CSV file with all student grades
 - Import this into your learning management system if needed

Managing Multiple Assignments Throughout the Semester

Step 12: Monitor Progress Across Assignments

1. Use the GitHub Classroom dashboard to:
 - Track which students have accepted each assignment
 - Monitor completion rates
 - Review autograding results
2. Set up notifications:
 - Go to your organization settings
 - Click on “Notifications”
 - Configure notification settings for assignment activities

Step 13: Release Assignment Feedback in Batch

1. After grading all submissions for an assignment:
 - Consider releasing all feedback at once
 - Announce to students that feedback is available
 - Remind them how to view their feedback (via repository comments)
2. Address common issues:
 - Create a “Common Issues” document addressing recurring problems
 - Share this with the class after grading

Step 14: Manage Late Submissions

1. Determine your late policy in advance
2. For late submissions:
 - Students can still push to their repositories after the deadline
 - GitHub tracks the timestamp of all commits
 - Review the commit history to verify submission timing
 - Apply your late policy when grading
3. To handle extensions:
 - Inform students they should continue working in the same repository

- Make note of approved extensions for grading purposes

Using GitHub Classroom Effectively with TAs

Step 15: Divide Grading Responsibilities with TAs

1. Assign specific students or assignments to each TA:
 - Create a spreadsheet mapping TAs to student subsets
 - Or assign different assignments to different TAs
2. Establish a grading process:
 - Define expectations for feedback detail and format
 - Set internal deadlines for completing grading
 - Create a grading rubric for consistency

Step 16: Ensure Grading Consistency

1. Create detailed rubrics for each assignment:
 - Break down point allocations
 - Define criteria for each score level
 - Share with all TAs
2. Conduct calibration sessions:
 - Before grading begins, select 2-3 sample submissions
 - Have all TAs independently grade them
 - Meet to discuss and align on expectations
3. Perform periodic reviews:
 - Randomly check a few submissions graded by each TA
 - Provide feedback to TAs on their grading

Step 17: Set Up TA Workflow

1. Create a private repository for teaching staff:
 - Store rubrics, answer keys, and grading notes
 - Document common issues and solutions
2. Establish a communication channel:
 - Use GitHub Discussions in the private repo
 - Or use another tool like Slack or Microsoft Teams
3. Set up regular check-ins:
 - Schedule weekly meetings to discuss grading progress
 - Address any questions or issues with assignments

End-of-Semester Tasks

Step 18: Archive and Back Up All Student Repositories

1. Use GitHub Classroom's download feature:
 - Go to the classroom dashboard
 - Click on "Download" for each assignment

- This downloads a ZIP file with all student repositories
2. Alternatively, clone all repositories:
 - Create a script to clone all repos for backup
 - Store these in a secure location following your institution's data policies

Step 19: Generate Final Grade Reports

1. Export all assignment grades:
 - Download grades for each assignment
 - Combine into a master spreadsheet
2. Calculate final grades according to your syllabus
3. Provide final feedback to students:
 - Consider creating a custom feedback document for each student
 - Highlight strengths and growth throughout the course

Step 20: Prepare for Next Semester

1. Review what worked and what didn't:
 - Which assignments were most effective?
 - Were the tests appropriate and accurate?
 - Were the instructions clear?
2. Archive the classroom:
 - In GitHub Classroom, you can archive a classroom when it's complete
 - This preserves all data but moves it out of your active classrooms
3. Create a new classroom for the next semester:
 - Use your improved assignments and tests
 - Update dates and details for the new semester

Troubleshooting Common Issues

Common Student Issues

1. **Student can't accept an assignment**
 - Verify they're logged in to the correct GitHub account
 - Check if they've already accepted the assignment
 - Regenerate the invitation link if needed
2. **Student pushed to the wrong branch**
 - Show them how to create a new branch with the correct content
 - Or help them merge changes to the main branch
3. **Student can't see autograding results**
 - Check the Actions tab in their repository
 - Verify the workflow ran successfully
 - Help them trigger a new run if needed

Common Instructor/TA Issues

1. **Autograding not working**
 - Check the test script for errors
 - Verify the GitHub Actions workflow file is correct
 - Test with a sample solution
2. **Can't see all student repositories**
 - Verify you have the correct permissions in the organization
 - Check that students have accepted the assignment
3. **Need to update an assignment after release**
 - Push changes to the starter code repository
 - Inform students they need to pull changes
 - For major changes, consider creating a new assignment

Bonus: Advanced GitHub Classroom Features

Using GitHub Classroom Assistant

1. Install GitHub Classroom Assistant:
 - Go to GitHub Classroom Assistant
 - Download the appropriate version for your system
2. Use it to clone all student repositories at once
3. Use it to push feedback to all repositories at once

Setting Up Group Assignments

For projects or pair programming: 1. Follow the same steps as individual assignments 2. Select “Group assignment” instead of “Individual assignment” 3. Configure team settings: - Set maximum team size - Allow or disallow self-formation of teams - Set up team naming convention

Integrating with Learning Management Systems

If your university uses Canvas, Moodle, or another LMS: 1. In GitHub Classroom, go to classroom settings 2. Select “Connect to a learning management system” 3. Follow the integration steps for your specific LMS 4. This allows grades to automatically sync between systems